UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE FÍSICA DE SÃO CARLOS

THIAGO OLIVEIRA DOS SANTOS

# Modelagem tridimensional de orbitais atômicos para visualização holográfica em Blender e Unity

São Carlos
2026

Trabalho de Conclusão de Curso – Monografia

Física Computacional

THIAGO OLIVEIRA DOS SANTOS

# Modelagem tridimensional de orbitais atômicos para visualização holográfica em Blender e Unity

Trabalho de conclusão de curso apresentado ao Instituto de Física de São Carlos da Universidade de São Paulo para obtenção do título de Bacharel em Física Computacional.

Orientador: Prof. Dr. Guilherme Matos Sipahi - Instituto de Física de São Carlos

São Carlos
2026

AUTORIZO A REPRODUÇÃO E DIVULGAÇÃO TOTAL OU PARCIAL DESTE TRABALHO, POR QUALQUER MEIO CONVENCIONAL OU ELETRÔNICO PARA FINS DE ESTUDO E PESQUISA, DESDE QUE CITADA A FONTE.

# AGRADECIMENTOS

Ao Professor Dr. Guilherme Matos Sipahi, meu orientador, pela oportunidade de desenvolver este trabalho, pela paciência e pelo incentivo ao longo destes dois semestres e pelo entusiasmo com que promove e organiza eventos de divulgação científica.

A toda a equipe do Estúdio de Mídia, Cultura e Ciência, pelas contribuições e pelo apoio sempre que encontrei dificuldades.

A todos os professores que fizeram parte da minha trajetória na graduação e contribuíram para minha formação crítica e profissional. Um agradecimento especial ao Professor Dr. Fernando Fernandes Paiva, meu orientador de Iniciação Científica, pela oportunidade, pelos conselhos e por todo o apoio que me ofereceu sempre que precisei.

A todos os colegas de classe e de curso com quem tive o prazer de realizar trabalhos ou compartilhar boas risadas nos corredores dos institutos. A todos os amigos que fiz ao longo desses anos, em especial ao Artur e ao João Pedro, por estarem ao meu lado nos momentos mais difíceis, por me escutarem e por me incentivarem a não desistir.

Ao Henrique e ao Bruno, meus colegas de quarto, por todo o apoio e carinho, por acompanharem cada noite de sono perdida, cada derrota e cada conquista ao longo dessa trajetória. Pelas conversas filosóficas e por sempre estarem dispostos a ouvir meus desabafos.

A todos os meus amigos de longa data, que sempre acreditaram em mim e me apoiaram, mesmo à distância. A todos que tive a oportunidade de conhecer ao longo desses anos, pelas experiências que me proporcionaram.

Ao meu pai, à minha mãe e às minhas irmãs, pelo amor incondicional, mesmo à distância e nos momentos em que não pude estar tão presente quanto gostaria. Vocês sempre acreditaram no meu potencial e se desdobraram para me ajudar sempre que precisei. Não consigo expressar em palavras o quanto sou grato por poder chamar vocês de família.

E, por fim, a Deus, por me permitir concluir mais esta etapa da minha vida.

# RESUMO

Este trabalho apresenta um fluxo computacional automatizado para gerar representações tridimensionais de orbitais do hidrogênio e animações GIF destinadas à visualização em um projetor de LEDs rotativos. As funções de onda e suas densidades de probabilidade são avaliadas em uma grade cartesiana tridimensional, nas bases real e complexa. O processamento integra Python para o cálculo científico e a extração das geometrias, Blender para a preparação dos modelos e Unity para a apresentação e a gravação, com orquestração pelo Node-RED. Foram implementados cinco modos de representação, baseados em isosuperfícies, múltiplos níveis de densidade, nuvens de pontos e codificação de sinal ou fase por cores. A avaliação dos estados com números quânticos principais de $n = 1$ a $n = 4$ confirmou a produção dos $240$ GIFs previstos para as combinações compatíveis de orbitais, bases e representações. A comparação qualitativa do orbital $(n, \ell, m) = (4, 2, 2)$ evidenciou informações complementares entre os modos e diferenças espaciais entre as funções selecionadas nas duas bases. Os resultados demonstram a execução integrada do processamento em lote, com registro dos parâmetros e estados de cada execução. A verificação realizada compreende a geração dos arquivos e sua inspeção no ambiente computacional.

**Palavras-chave:** Orbitais atômicos. Visualização científica. Modelagem tridimensional.

# ABSTRACT

This work presents an automated computational workflow for generating three-dimensional representations of hydrogen atomic orbitals and GIF animations intended for visualization on a rotating LED display. Wavefunctions and their probability densities are evaluated on a three-dimensional Cartesian grid using real and complex bases. The workflow integrates Python for scientific computation and geometry extraction, Blender for model preparation, and Unity for presentation and recording, with orchestration through Node-RED. Five representation modes were implemented, based on isosurfaces, multiple density levels, point clouds, and color coding of wavefunction sign or phase. The evaluation of states with principal quantum numbers from $n = 1$ to $n = 4$ confirmed the generation of the $240$ GIF files expected for the compatible combinations of orbitals, bases, and representations. A qualitative comparison of the orbital $(n, \ell, m) = (4, 2, 2)$ highlighted complementary information across representation modes and spatial differences between the functions selected in the two bases. The results demonstrate integrated batch processing, with parameters and execution states recorded for each run. The verification covers file generation and inspection within the computational environment.

**Keywords:** Atomic orbitals. Scientific visualization. Three-dimensional modeling.

# SUMÁRIO

**1 Introdução** **1**

**2 Fundamentação teórica** **1**

2.1 Funções de onda, estados estacionários e autovalores . . . . . 2

2.2 Átomo de hidrogênio e orbitais atômicos . . . . . 2

2.3 Harmônicos esféricos e bases real e complexa . . . . . 3

2.4 Densidade de probabilidade, sinal e fase . . . . . 4

**3 Materiais e métodos** **5**

3.1 Ferramentas computacionais . . . . . 5

3.2 Modelagem computacional dos orbitais . . . . . 6

3.3 Representações dos orbitais e compatibilidade entre bases . . . . . 7

3.4 Preparação dos modelos e animação . . . . . 8

3.5 Configuração das execuções e critérios de verificação . . . . . 11

**4 Arquitetura e implementação** **12**

4.1 Separação das etapas e formatos de intercâmbio . . . . . 12

4.2 Organização interna das etapas . . . . . 13

4.3 Configuração de Jobs e rastreabilidade das Runs . . . . . 16

4.4 Orquestração, validação e retomada . . . . . 17

**5 Resultados e discussão** **19**

5.1 Execução em lote e cobertura das representações . . . . . 19

5.2 Comparação das representações e das bases . . . . . 20

5.3 Animações e limitações da visualização . . . . . 21

**6 Conclusões e considerações finais** **23**

**Referências** **24**

# 1 INTRODUÇÃO

A visualização de orbitais atômicos permite relacionar a descrição matemática dos estados quânticos à sua estrutura espacial. Essa relação envolve a distribuição da densidade de probabilidade e as informações de sinal e fase da função de onda. Imagens estáticas podem encobrir estruturas internas e dificultar a percepção de sua disposição tridimensional, motivando o uso de diferentes representações e orientações de observação [1].

Entre as iniciativas voltadas a essa visualização, Hanson apresenta uma ferramenta em Python para explorar distribuições tridimensionais de densidade dos orbitais do hidrogênio [1]. No contexto do Instituto de Física de São Carlos, Graça desenvolveu um jogo em realidade virtual com representações tridimensionais de funções de onda e interação pelo Unity [2]. Esses trabalhos exemplificam o uso de recursos computacionais para apresentar conceitos quânticos por meio de objetos e imagens.

A produção de conjuntos de visualizações exige coordenar o cálculo dos campos, a construção das geometrias e sua apresentação. Quando diferentes orbitais, bases e representações são considerados, torna-se necessário organizar os parâmetros e os arquivos produzidos em cada etapa. Neste projeto, essa necessidade está associada à preparação de animações destinadas à exibição em um projetor de LEDs rotativos.

O objetivo deste trabalho é desenvolver um fluxo computacional automatizado para modelar orbitais do hidrogênio e gerar animações em formato GIF, integrando Python, Blender e Unity, com orquestração pelo Node-RED. Para isso, são avaliadas as funções de onda e suas densidades, construídas as representações geométricas, preparados os modelos e gravadas suas rotações. Os parâmetros e os estados das execuções são registrados para permitir o acompanhamento do processamento e a associação entre as configurações e as saídas.

A avaliação abrange estados estacionários com $n = 1, \ldots, 4$, nas bases real e complexa, e os modos `single`, `signed`, `multilayer`, `point_cloud` e `phase`, respeitando sua compatibilidade com cada base. A verificação considera a cobertura dos arquivos solicitados e a comparação qualitativa das representações e animações no ambiente computacional, tendo a exibição no projetor como aplicação prevista para o material produzido.

# 2 FUNDAMENTAÇÃO TEÓRICA

A construção das representações tridimensionais dos orbitais exige relacionar as soluções estacionárias da equação de Schrödinger às grandezas que serão visualizadas. Para o átomo de hidrogênio, essa relação envolve a estrutura radial e angular da função de onda, sua expressão em bases reais ou complexas e a interpretação da densidade de probabilidade, do sinal e da fase. Esses conceitos fundamentam as escolhas de geometria e cor adotadas no trabalho.

## 2.1 Funções de onda, estados estacionários e autovalores

Fenômenos como a difração e a interferência evidenciam o comportamento ondulatório da matéria, enquanto a detecção individual de partículas produz eventos localizados. Na formulação ondulatória da mecânica quântica, o estado de uma partícula é descrito pela função de onda $\Psi(\mathbf{r}, t)$, definida em termos da posição $\mathbf{r}$ e do tempo $t$. Essa função, em geral complexa, contém as informações utilizadas para prever os resultados possíveis de medições e suas probabilidades, e sua evolução temporal é determinada pela equação de Schrödinger [3].

A relação entre a função de onda e as medidas de posição é estabelecida pela interpretação estatística de Born, segundo a qual $|\Psi(\mathbf{r}, t)|^2$ fornece a densidade de probabilidade de posição. Para um Hamiltoniano independente do tempo, os estados estacionários são obtidos das soluções da equação de Schrödinger independente do tempo,

$$\hat{H}\psi(\mathbf{r}) = E\psi(\mathbf{r}), \tag{2.1}$$

na qual $\hat{H}$ é o operador Hamiltoniano, associado à energia total do sistema, $\psi(\mathbf{r})$ é a parte espacial da função de onda e $E$ é a energia correspondente. Nessa relação, $\psi$ é uma autofunção do Hamiltoniano e representa um autoestado de energia, enquanto $E$ é seu autovalor. Uma medida da energia nesse estado fornece o valor $E$.

A dependência temporal de um estado estacionário é dada por

$$\Psi(\mathbf{r}, t) = \psi(\mathbf{r})e^{-iEt/\hbar}, \tag{2.2}$$

em que $\hbar$ é a constante de Planck reduzida. Como o fator exponencial tem módulo unitário, a densidade de probabilidade satisfaz

$$|\Psi(\mathbf{r}, t)|^2 = |\psi(\mathbf{r})|^2 . \tag{2.3}$$

Esses estados são denominados estacionários porque sua densidade de probabilidade permanece constante no tempo, embora a função de onda possua uma fase temporal. Essa propriedade permite construir as representações dos orbitais a partir da função espacial $\psi(\mathbf{r})$.

## 2.2 Átomo de hidrogênio e orbitais atômicos

O átomo de hidrogênio é constituído por um elétron ligado a um próton pela interação eletrostática. No tratamento não relativístico, considerando o próton fixo na origem, o Hamiltoniano é dado por [3]

$$\hat{H} = -\frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2 - \frac{e^2}{4\pi\varepsilon_0 r}, \tag{2.4}$$

em que $m_\mathrm{e}$ é a massa do elétron, $e$ é o módulo da carga elementar, $\varepsilon_0$ é a permissividade elétrica do vácuo e $r$ é a distância entre o elétron e o próton. O primeiro termo representa a energia cinética e o segundo corresponde à energia potencial coulombiana.

Como o potencial depende apenas de $r$, a Eq. (2.1) admite separação de variáveis em coordenadas esféricas. As soluções espaciais dos estados ligados podem ser escritas na forma

$$\psi_{n\ell m}(r,\theta,\phi) = R_{n\ell}(r)Y_\ell^m(\theta,\phi), \tag{2.5}$$

na qual $R_{n\ell}$ é a função radial e $Y_\ell^m$ é um harmônico esférico, responsável pela dependência angular. Os ângulos $\theta$ e $\phi$ são, respectivamente, o ângulo polar e o azimutal. Essas funções de onda espaciais constituem os orbitais atômicos do hidrogênio.

Os estados são classificados pelos números quânticos principal $n$, orbital $\ell$ e magnético $m$, cujos valores permitidos são $n = 1, 2, \ldots$, $\ell = 0, 1, \ldots, n-1$ e $m = -\ell, -\ell+1, \ldots, \ell$. No modelo considerado, a energia depende apenas de $n$, segundo $E_n \approx -13{,}6\,\mathrm{eV}/n^2$. Assim, diferentes combinações de $\ell$ e $m$ podem corresponder à mesma energia [3].

A escala de comprimento das soluções é estabelecida pelo raio de Bohr, $a_0 = 4\pi\varepsilon_0\hbar^2/(m_\mathrm{e}e^2)$. Na convenção dos polinômios generalizados de Laguerre empregada neste trabalho, a função radial normalizada pode ser escrita como [1]

$$R_{n\ell}(r) = \sqrt{\left(\frac{2}{na_0}\right)^3 \frac{(n-\ell-1)!}{2n(n+\ell)!}} e^{-r/(na_0)} \left(\frac{2r}{na_0}\right)^\ell L_{n-\ell-1}^{2\ell+1}\left(\frac{2r}{na_0}\right), \tag{2.6}$$

em que $L_{n-\ell-1}^{2\ell+1}$ é um polinômio generalizado de Laguerre de grau $n-\ell-1$. A combinação das partes radial e angular determina a distribuição espacial da função de onda e, consequentemente, a estrutura das representações tridimensionais dos orbitais.

## 2.3 Harmônicos esféricos e bases real e complexa

Os harmônicos esféricos descrevem a dependência angular dos orbitais na Eq. (2.5). Para $m \geq 0$, sua forma complexa pode ser expressa como

$$Y_\ell^m(\theta,\phi) = \sqrt{\frac{2\ell+1}{4\pi}\frac{(\ell-m)!}{(\ell+m)!}} P_\ell^m(\cos\theta) e^{im\phi}, \tag{2.7}$$

em que $P_\ell^m$ é uma função associada de Legendre. Adota-se aqui a convenção na qual o fator de fase de Condon–Shortley, $(-1)^m$, está incluído em $P_\ell^m$. Os harmônicos de índices opostos satisfazem $Y_\ell^{-m} = (-1)^m (Y_\ell^m)^*$, em que o asterisco indica conjugação complexa [3].

Uma base de funções reais pode ser construída por combinações lineares dos harmônicos complexos de índices $m$ e $-m$. Essas combinações preservam a ortonormalidade e descrevem o mesmo

espaço de estados angulares para cada valor de $\ell$ [1]. Neste trabalho, os harmônicos reais são definidos pela convenção

$$\mathcal{Y}_{\ell m} = \begin{cases} \sqrt{2}(-1)^{|m|}\,\mathrm{Im}\left[Y_\ell^{|m|}\right], & m < 0, \\ Y_\ell^0, & m = 0, \\ \sqrt{2}(-1)^m\,\mathrm{Re}[Y_\ell^m], & m > 0, \end{cases} \tag{2.8}$$

com os argumentos $(\theta, \phi)$ omitidos por simplicidade. Os símbolos $\mathrm{Re}$ e $\mathrm{Im}$ indicam as partes real e imaginária. A função de onda na base real é obtida substituindo $Y_\ell^m$ por $\mathcal{Y}_{\ell m}$ na Eq. (2.5).

Na base complexa, $m$ está associado ao autovalor $m\hbar$ da componente $z$ do momento angular orbital. Na base real, para $m \neq 0$, esse índice identifica uma combinação de estados com componentes opostas do momento angular. Essas combinações permanecem autoestados de energia, pois os estados envolvidos possuem a mesma energia.

É necessário distinguir a mudança de base de um mesmo estado da escolha de funções diferentes em cada base. Ao reescrever um estado em outra base, seus coeficientes também são transformados, preservando as previsões físicas. Já a seleção de um único orbital com o mesmo rótulo $(n, \ell, m)$ nas duas bases pode representar estados distintos quando $m \neq 0$, produzindo diferentes distribuições espaciais de probabilidade. Essa distinção orienta a comparação visual das representações desenvolvidas neste trabalho.

## 2.4 Densidade de probabilidade, sinal e fase

Para os estados estacionários considerados neste trabalho, a densidade de probabilidade de posição é dada por $\rho(\mathbf{r}) = |\psi(\mathbf{r})|^2$. Essa densidade não representa, isoladamente, uma probabilidade. A probabilidade, em si, de se encontrar o elétron em uma região $\Omega$ é obtida pela integral

$$P(\Omega) = \int_\Omega |\psi(\mathbf{r})|^2\, d^3r. \tag{2.9}$$

Como a probabilidade de encontrar a partícula em todo o espaço deve ser unitária, a função de onda deve satisfazer a condição de normalização [3],

$$\int_{\mathbb{R}^3} |\psi(\mathbf{r})|^2\, d^3r = 1. \tag{2.10}$$

Além do módulo, uma função de onda complexa possui uma fase. Nos pontos em que $\psi(\mathbf{r}) \neq 0$, pode-se escrever

$$\psi(\mathbf{r}) = |\psi(\mathbf{r})|e^{i\chi(\mathbf{r})}, \qquad \chi(\mathbf{r}) = \arg\psi(\mathbf{r}), \tag{2.11}$$

em que $\chi$ é a fase espacial. Nos nós, onde a função de onda se anula, essa fase não é definida. Para uma função real, os valores positivos e negativos correspondem, respectivamente, às fases $0$ e $\pi$,

módulo $2\pi$. Portanto, o sinal distingue regiões da função de onda, sem representar probabilidades positivas ou negativas.

É necessário distinguir essa variação espacial de uma fase global. Multiplicar toda a função de onda por $e^{i\chi_0}$, com $\chi_0$ constante, preserva o estado físico e, em particular, a densidade de probabilidade [3]:

$$\left|e^{i\chi_0}\psi(\mathbf{r})\right|^2 = |\psi(\mathbf{r})|^2. \tag{2.12}$$

Assim, inverter simultaneamente todos os sinais de uma função real também preserva o estado físico. A interpretação das cores depende, portanto, de uma convenção de fase mantida entre as representações.

Neste trabalho, a densidade de probabilidade, o sinal das funções reais e a fase das funções complexas orientam as escolhas de geometria e cor. As cores associadas ao sinal ou à fase permitem distinguir informações que o módulo ao quadrado, isoladamente, não apresenta. A construção dessas representações é descrita na seção de métodos.

# 3 MATERIAIS E MÉTODOS

Esta seção descreve o percurso computacional entre a avaliação das funções de onda dos orbitais de hidrogênio e a geração das animações em formato GIF destinadas ao projetor. Inicialmente, apresentam-se as ferramentas utilizadas e a amostragem das funções de onda em uma grade tridimensional. Em seguida, são descritas as representações construídas a partir desses dados, a preparação dos modelos no Blender e sua apresentação e gravação no Unity. Por fim, são definidos os parâmetros das execuções e os critérios utilizados para verificar a conclusão das etapas e os arquivos produzidos.

## 3.1 Ferramentas computacionais

A etapa de cálculo científico foi desenvolvida em Python. A biblioteca NumPy foi utilizada para construir a grade tridimensional e realizar operações sobre os arrays que armazenam as coordenadas e os valores dos campos [4]. A SciPy forneceu as funções especiais necessárias à avaliação dos polinômios generalizados de Laguerre e dos harmônicos esféricos [5]. Para extrair malhas de isosuperfícies dos campos amostrados, empregou-se a implementação de Marching Cubes disponível na biblioteca scikit-image [6].

O Blender foi utilizado na preparação dos modelos tridimensionais, incluindo a importação das geometrias, a aplicação de materiais e a organização dos objetos em uma cena. Os modelos destinados à etapa seguinte foram exportados no formato FBX [7].

A apresentação dos modelos e a geração das animações foram realizadas no Unity, versão `6000.5.7f1` [8], com scripts desenvolvidos em C#. O pacote Unity Recorder, versão `5.1.7`, foi empregado para capturar a visualização da câmera e produzir os arquivos GIF [9].

A execução automatizada foi organizada por scripts coordenadores, denominados *runners*, responsáveis por conduzir as operações de cada etapa. O projeto também disponibiliza uma interface de linha de comando (CLI, do inglês *command-line interface*), que permite executar operações por comandos textuais e informar seus parâmetros. A integração entre as etapas foi realizada com o Node-RED, versão `5.0.9`, por meio de um fluxo visual de orquestração [10].

O código-fonte foi mantido sob controle de versões com Git e hospedado no GitHub. O acesso ao repositório e aos registros da execução piloto é apresentado na subseção 4.3.

## 3.2 Modelagem computacional dos orbitais

A modelagem computacional consiste na avaliação das expressões analíticas dos orbitais em uma grade tridimensional. Para cada combinação de números quânticos $(n, \ell, m)$ e base escolhida, calculam-se os valores da função de onda e da densidade de probabilidade nos pontos dessa grade.

As coordenadas espaciais são expressas em unidades do raio de Bohr, adotando-se $a_0 = 1$ no cálculo. O domínio utilizado é um cubo centrado no núcleo, $[-L_n, L_n]^3$, cuja extensão depende do número quântico principal. Na implementação, o semicomprimento do domínio e o espaçamento da grade são definidos por

$$L_n = \big[5 + 10(n-1)\big]\, a_0, \qquad \Delta = \frac{2L_n}{N-1}, \tag{3.1}$$

em que $N$ é o número de pontos por eixo, incluindo os extremos [11]. A grade contém, portanto, $N^3$ pontos. A extensão do domínio estabelece a região espacial representada, enquanto o espaçamento determina a resolução da amostragem. Para um mesmo $N$, aumentar o domínio também aumenta a distância entre os pontos. Os valores empregados nas execuções são apresentados na subseção 3.5.

Para avaliar as funções radial e angular, as coordenadas cartesianas de cada ponto são convertidas em coordenadas esféricas:

$$r = \sqrt{x^2 + y^2 + z^2}, \qquad \theta = \arccos\left(\frac{z}{r}\right), \qquad \phi = \operatorname{atan2}(y, x). \tag{3.2}$$

A função $\operatorname{atan2}$ determina o ângulo azimutal considerando o quadrante do ponto. Quando a origem pertence à grade, os ângulos nesse ponto são definidos como zero, evitando a divisão por zero no cálculo de $\theta$.

A função radial é calculada pela Eq. (2.6), enquanto a parte angular segue a base complexa ou a combinação real definida na Eq. (2.8). Em cada ponto $\mathbf{r}_{ijk}$, armazenam-se

$$\psi_{ijk} = \psi(\mathbf{r}_{ijk}), \qquad \rho_{ijk} = |\psi_{ijk}|^2, \tag{3.3}$$

com $i, j, k = 0, \ldots, N-1$. São mantidos os fatores de normalização das expressões analíticas; o domínio finito limita a parcela espacial dessas funções incluída na visualização.

Os arrays resultantes preservam a informação da função de onda, real ou complexa, e sua densidade de probabilidade. Esses campos constituem a entrada para a construção das geometrias e para a atribuição das cores descritas na próxima subseção.

## 3.3 Representações dos orbitais e compatibilidade entre bases

Uma isosuperfície é o conjunto de pontos nos quais um campo escalar assume um valor constante, denominado nível ou limiar. Para as representações baseadas na densidade de probabilidade, define-se

$$\tau = p\rho_{\max}, \qquad 0 < p < 1, \tag{3.4}$$

em que $\rho_{\max}$ é o maior valor de densidade encontrado na grade e $p$ é uma fração configurável para cada modo. Esse parâmetro estabelece um nível relativo ao máximo amostrado; não corresponde à probabilidade integrada no interior da superfície.

A extração das isosuperfícies é realizada por Marching Cubes, que percorre as células da grade e constrói uma aproximação da superfície por triângulos [12, 6]. A Tabela 1 apresenta os cinco modos implementados e sua disponibilidade em cada base.

Tabela 1: Representações implementadas e compatibilidade com as bases real e complexa.

| Modo | Informação representada | Real | Complexa |
|---|---|---|---|
| `single` | Uma isosuperfície de densidade | Sim | Sim |
| `signed` | Sinal da função de onda real | Sim | Não |
| `multilayer` | Vários níveis de densidade | Sim | Sim |
| `point_cloud` | Pontos em uma faixa de densidade | Sim | Sim |
| `phase` | Fase sobre uma isosuperfície de densidade | Não | Sim |

Fonte: Elaborado pelo autor

No modo `single`, extrai-se uma única isosuperfície definida por $\rho(\mathbf{r}) = \tau$. O modo `signed`, por sua vez, distingue as regiões positivas e negativas de uma função de onda real. Para isso, são extraídas superfícies nos níveis

$$\psi(\mathbf{r}) = \pm\sqrt{\tau}, \tag{3.5}$$

quando esses níveis estão presentes no campo. No Blender, o sinal positivo recebe a cor vermelha e o negativo, a azul, segundo a convenção adotada no trabalho.

O modo `multilayer` utiliza $K$ isosuperfícies com níveis igualmente espaçados:

$$\tau_q = \rho_{\max}\left[p + \frac{q}{K}(1-p)\right], \qquad q = 0, \ldots, K-1. \tag{3.6}$$

Assim, o primeiro nível é $p\rho_{\max}$ e os demais aumentam uniformemente, permanecendo abaixo de $\rho_{\max}$. Não se extrai uma isosuperfície exatamente no valor máximo da densidade. As camadas recebem cores e transparências diferentes conforme o nível, com maior transparência nos níveis mais baixos.

No modo `point_cloud`, selecionam-se os pontos da grade que satisfazem

$$|\rho_{ijk} - \tau| < \varepsilon \rho_{\max}, \tag{3.7}$$

em que $\varepsilon > 0$ controla a tolerância ao redor do nível escolhido. A seleção é determinística, e a quantidade de pontos depende do limiar, da tolerância e da resolução da grade. No Blender, esses pontos são representados por pequenos octaedros.

O modo `phase` parte da mesma isosuperfície de densidade utilizada em `single`. As partes real e imaginária da função de onda são interpoladas nos centros dos triângulos, permitindo calcular a fase local. O intervalo $[0, 2\pi)$ é dividido em $B$ faixas iguais, e cada face recebe uma cor de uma escala cíclica conforme sua faixa de fase.

Os modos `single`, `multilayer` e `point_cloud` utilizam somente a densidade, podendo operar nas duas bases. O modo `signed` requer uma função real para distinguir seus sinais. Já a disponibilidade de `phase` apenas na base complexa é uma escolha da implementação: funções reais também possuem fases, conforme discutido na fundamentação teórica.

As geometrias produzidas por esses critérios são encaminhadas ao Blender para a preparação dos modelos e, posteriormente, ao Unity para a apresentação e a gravação das animações.

## 3.4 Preparação dos modelos e animação

No fluxo automatizado, denomina-se *Job* o pedido de geração, que especifica os orbitais, as bases, as representações e os parâmetros de processamento. Uma *Run* corresponde a uma execução desse pedido, reunindo seus registros e arquivos produzidos.

O Blender recebe as malhas no formato OBJ e, para o modo `point_cloud`, arquivos textuais com coordenadas separadas por vírgula, que utilizam a extensão `.npy` na implementação. Os dados são convertidos em objetos da cena, com os materiais correspondentes a cada representação, e exportados como modelos FBX.

A Figura 1 apresenta uma cena de demonstração com orbitais de $n = 1$ e $n = 2$. Quando o salvamento da cena é solicitado, o *runner* organiza os modelos para inspeção e grava um arquivo `.blend` após exportar os FBXs. Para a captura apresentada, os objetos foram rearranjados manualmente.

No Unity, os modelos FBX importados compõem um catálogo identificado pelos números quânticos, pela base e pelo modo de representação. A seleção de uma entrada instancia o modelo na cena e aplica os parâmetros de apresentação. Na operação pelo Editor, esses parâmetros podem ser inspecionados no *Inspector*, painel de propriedades dos objetos e componentes. A *Game View* mostra a imagem produzida pela câmera, utilizada na gravação.

A Figura 2 exemplifica essa organização para o orbital $(n, \ell, m) = (4, 2, 2)$, na base real e no modo `signed`.

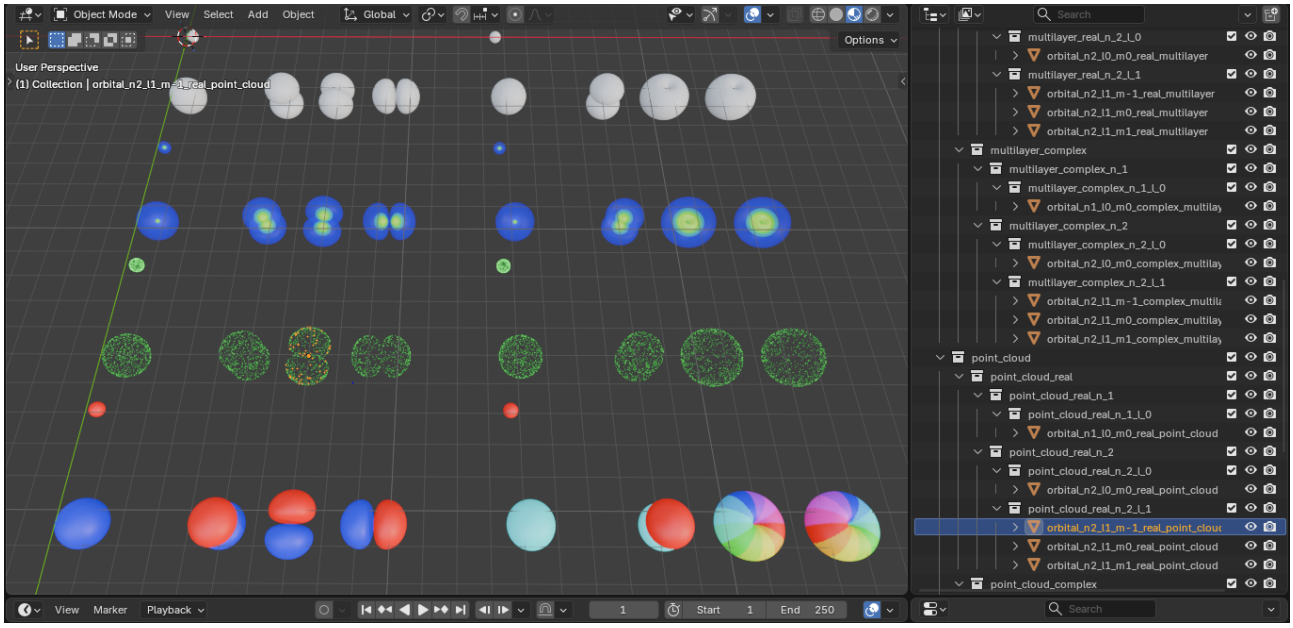



Figura 1: Cena de demonstração no Blender com orbitais de $n = 1$ e $n = 2$. Os modelos foram rearranjados manualmente para a captura. À direita, o Outliner mostra parte da hierarquia de coleções.

Fonte: Elaborado pelo autor

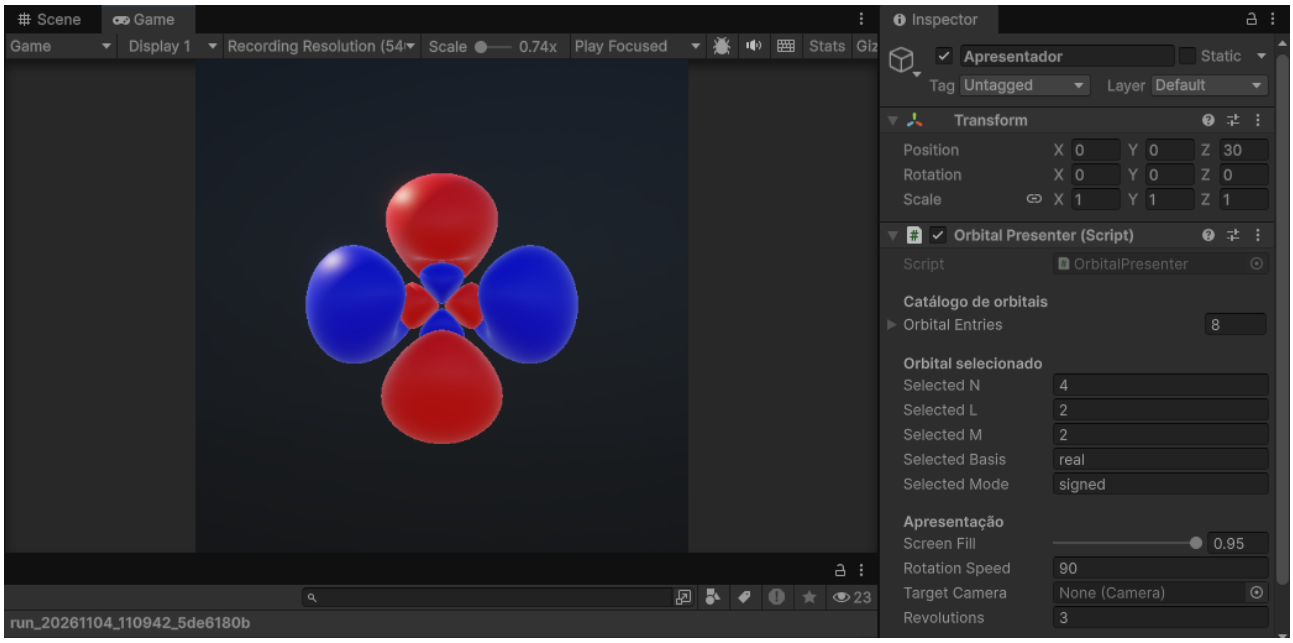



Figura 2: Interface do Unity com o orbital $(n, \ell, m) = (4, 2, 2)$, na base real e no modo `signed`, exibido na Game View. O Inspector mostra os parâmetros do componente `OrbitalPresenter`.

Fonte: Elaborado pelo autor

Para enquadrar o modelo, sua geometria é centralizada em um pivô e recebe uma escala calculada a partir da caixa envolvente e do campo de visão da câmera. O parâmetro `screen_fill` multiplica essa escala: valores maiores ampliam o modelo na imagem, enquanto valores menores aumentam a margem ao seu redor. Como o ajuste é realizado individualmente, o tamanho aparente dos diferentes orbitais não preserva uma escala espacial comum. A geometria desse cálculo é detalhada na subseção 4.2.

A resolução de gravação determina a quantidade de pixels de cada quadro. A Figura 3 ilustra os efeitos do enquadramento e da resolução para o mesmo orbital e a mesma pose. O quadro B provém da execução piloto, enquanto A e C foram obtidos em gravações dedicadas à comparação.

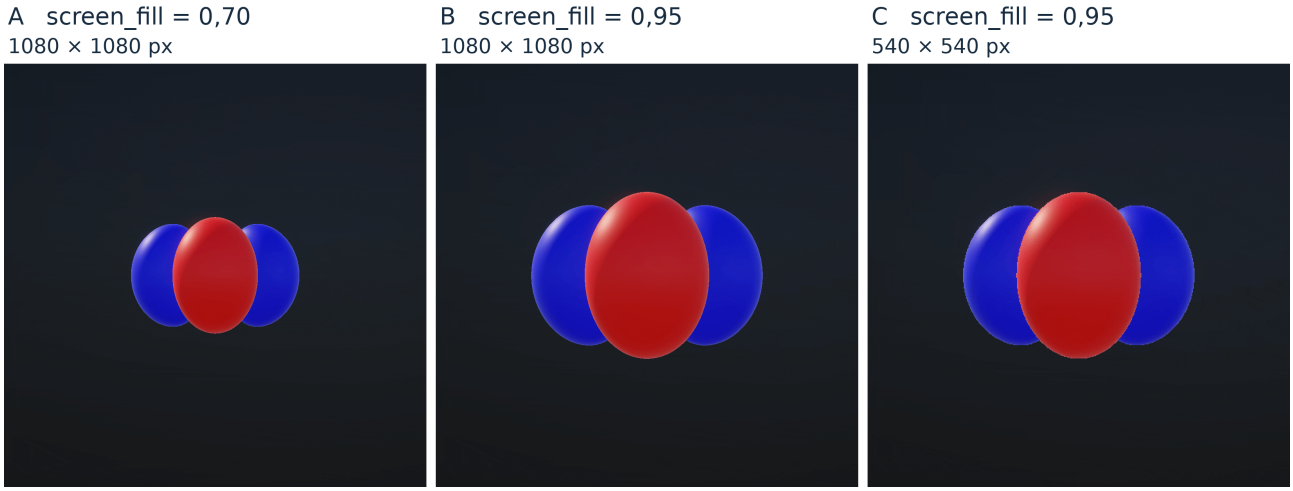



Figura 3: Comparação do enquadramento e da resolução para $(n, \ell, m) = (4, 2, 2)$, na base real e no modo `signed`. Entre A e B, varia-se `screen_fill`; entre B e C, varia-se a resolução. Os quadros iniciais dos GIFs são apresentados com a mesma largura.

Fonte: Elaborado pelo autor

Entre A e B, a alteração de `screen_fill` de $0{,}70$ para $0{,}95$ aumenta o tamanho aparente do modelo e reduz a margem ao redor dele. Entre B e C, compara-se a captura de $540 \times 540$ pixels com a de $1080 \times 1080$ pixels, mantendo `screen_fill` em $0{,}95$. Para a montagem, o quadro de menor resolução foi ampliado por vizinho mais próximo, tornando mais evidente a discretização das bordas.

A animação corresponde à rotação geométrica do modelo de um estado estacionário. O componente `Rotation` gira o pivô com velocidade angular constante, alternando os eixos $Y$, $X$ e $Z$ a cada volta. Para uma velocidade $\omega$, expressa em graus por segundo, e um total de $N_{\mathrm{v}}$ voltas, a duração programada é

$$T = \frac{360 N_{\mathrm{v}}}{\omega}. \tag{3.8}$$

O Unity Recorder captura a Game View e produz o GIF com a resolução, a taxa de quadros e os parâmetros de codificação definidos para a execução [9]. Esse procedimento pode ser aplicado a um orbital individual ou repetido para o conjunto de modelos selecionados.

## 3.5 Configuração das execuções e critérios de verificação

A configuração de cada execução é registrada no arquivo `job.json`, que reúne os parâmetros efetivos utilizados pelas etapas. O arquivo `run.json` identifica a Run e registra seu estado geral, o estado de cada etapa e eventuais erros. Esses registros permitem associar os arquivos produzidos às condições de geração, independentemente de alterações posteriores nos parâmetros padrão ou nos presets.

O lote completo corresponde à Run `run_20261104_114137_42180c82`. Sua seleção abrange todos os valores permitidos de $\ell$ e $m$ para $n = 1, \ldots, 4$, nas bases real e complexa, respeitando a compatibilidade dos modos apresentada na Tabela 1. Os parâmetros efetivos do lote são apresentados na Tabela 2.

Tabela 2: Parâmetros efetivos da execução do lote completo.

| Parâmetro | Valor |
|---|---|
| Pontos por eixo ($N$) | 300 |
| Fração de densidade ($p$) | 0,01 |
| Camadas de `multilayer` ($K$) | 10 |
| Tolerância de `point_cloud` ($\varepsilon$) | $5 \times 10^{-5}$ |
| Faixas de fase ($B$) | 32 |
| Escala global de exportação FBX | 1,0 |
| Velocidade angular | $90°/\mathrm{s}$ |
| Número de voltas | 3 |
| `screen_fill` | 0,95 |
| Resolução de gravação | $1080 \times 1080$ pixels |
| Taxa de quadros | 50 quadros/s |
| Qualidade do codificador GIF | 90 |
| Repetição do GIF | Ativada |

Fonte: Elaborado pelo autor

A comparação visual das representações utiliza a Run piloto `run_20261104_110942_5de6180b`, com o orbital $(n, \ell, m) = (4, 2, 2)$ nas duas bases. Os quadros do painel de representações foram extraídos no instante de $0{,}50$ s das animações, com recorte idêntico. A cena de demonstração no Blender, por sua vez, reúne orbitais de $n = 1$ e $n = 2$ para documentar sua organização e aparência.

Para a comparação da Figura 3, as gravações dedicadas A e C utilizam o orbital $(4, 2, 2)$, na base real e no modo `signed`, com $250$ pontos por eixo e $p = 0{,}01$. A corresponde à Run `run_20261105_041007_b4f2d7ec`, com `screen_fill` igual a $0{,}70$ e resolução de $1080 \times 1080$ pixels. C corresponde à Run `run_20261105_043243_fd7cebca`, com `screen_fill` igual a $0{,}95$ e resolução de $540 \times 540$ pixels. O quadro B provém do piloto, com `screen_fill` igual a $0{,}95$ e resolução de $1080 \times 1080$ pixels. As gravações utilizam $50$ quadros por segundo, velocidade angular de $90°/\mathrm{s}$ e três voltas.

A verificação do lote completo considera a conclusão das três etapas, identificadas como `scientific`, `blender` e `unity`, mediante o estado `completed` no `run.json`. Complementarmente, os GIFs finais são contados nas pastas de cada representação, excluindo-se o diretório temporário `_incomplete`. Conferem-se a presença de conteúdo nos arquivos, a conformidade dos nomes com os números quânticos, a base e o modo solicitados, e a correspondência entre as quantidades observadas e esperadas em cada grupo.

Esses critérios verificam a conclusão operacional do processamento e a cobertura das combinações solicitadas. A inspeção visual complementa essa verificação pela análise da geometria, das cores, do enquadramento e da rotação dos modelos. As contagens e a discussão das representações são apresentadas na seção de resultados.

# 4 ARQUITETURA E IMPLEMENTAÇÃO

A implementação organiza o processamento em etapas executadas nos ambientes de cálculo científico, preparação dos modelos e gravação das animações. Esta seção apresenta as responsabilidades de cada etapa e os arquivos utilizados na comunicação entre elas. Em seguida, descreve a organização interna dos módulos e componentes, a configuração dos Jobs e os registros que permitem acompanhar as Runs. Por fim, apresenta a orquestração pelo Node-RED, as validações que condicionam o avanço do processamento e os mecanismos de retomada de execuções interrompidas.

## 4.1 Separação das etapas e formatos de intercâmbio

O processamento foi dividido em três etapas: cálculo científico, preparação dos modelos e apresentação e gravação. Cada etapa é executada em um processo próprio, com as ferramentas necessárias às suas operações. A comunicação entre esses processos ocorre por arquivos, conforme a Figura 4.

A etapa científica utiliza o ambiente Python com NumPy, SciPy e scikit-image. A preparação dos modelos utiliza o interpretador Python interno do Blender e sua interface de programação, disponível pelo módulo `bpy`. No Unity, os componentes de apresentação e gravação são executados em C# no Editor, com acesso ao pacote Recorder. Essa divisão permite configurar as dependências de cada ambiente e executar as etapas individualmente.

Para cada orbital e base selecionados, a etapa científica calcula os campos amostrados e exporta a representação solicitada. Os modos baseados em superfícies produzem malhas OBJ, contendo vértices e faces. O modo `point_cloud` produz um arquivo textual com as coordenadas dos pontos selecionados.

O Blender recebe esses arquivos, constrói os objetos e aplica os materiais associados à representação. Nos modos compostos por várias partes, como `signed`, `multilayer` e `phase`, essas partes são reunidas no modelo correspondente. O resultado é exportado em FBX, levando a geometria e os materiais à etapa de apresentação.

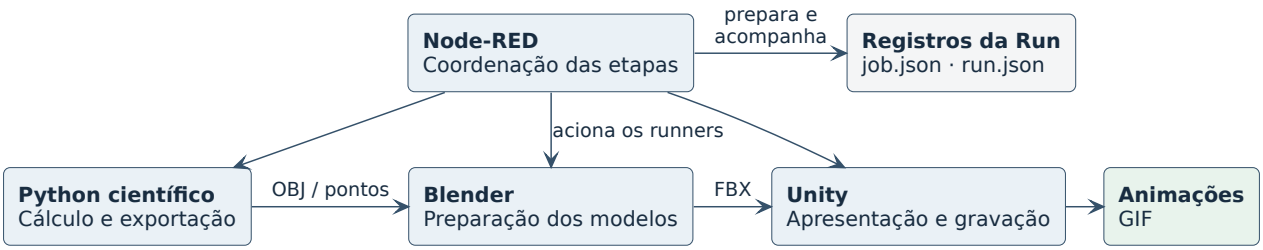



Figura 4: Etapas do pipeline, formatos de intercâmbio e registros de execução.

Fonte: Elaborado pelo autor

O Unity importa os FBXs e identifica os modelos pelos números quânticos, pela base e pelo modo. Cada modelo é instanciado, enquadrado e animado, e a imagem da câmera é capturada para gerar o GIF. Assim, a sequência de arquivos permite acompanhar uma representação desde a malha ou o conjunto de pontos até a animação final.

Na execução automatizada, o Node-RED aciona os runners e acompanha o andamento das etapas. O caminho da Run identifica o conjunto de arquivos utilizado no processamento: o `job.json` fornece os parâmetros efetivos, enquanto o `run.json` registra os estados de execução. A organização das pastas e a relação entre esses registros e as saídas são detalhadas na subseção 4.3.

## 4.2 Organização interna das etapas

Na implementação Python, as responsabilidades são distribuídas entre os módulos apresentados na Figura 5(a). Os `runners` coordenam a preparação e a execução das etapas; `config` resolve e valida as configurações; `services` reúne operações sobre orbitais e Runs; `core` contém a modelagem dos orbitais, da seleção e da grade; e `io` concentra operações de leitura, escrita e exportação.

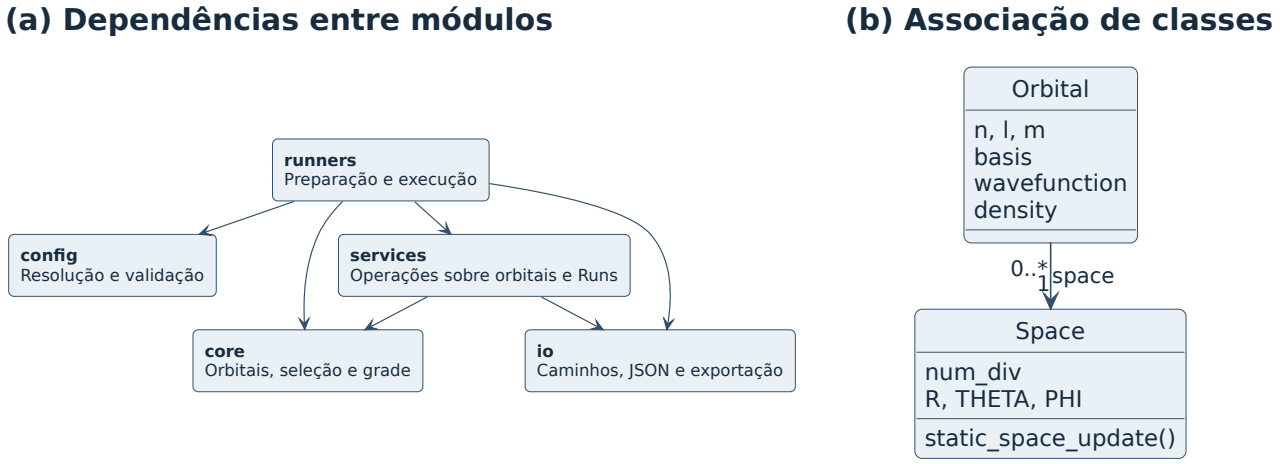



Figura 5: Dependências selecionadas entre os módulos Python e associação entre as classes `Orbital` e `Space`.

Fonte: Elaborado pelo autor

A Figura 5(b) destaca a associação entre `Orbital` e `Space`. A classe `Space` armazena a grade e suas coordenadas cartesianas e esféricas. Cada `Orbital` referencia um `Space` e reúne os números quânticos, a base e os arrays da função de onda e da densidade.

Na geração por intervalo da CLI, a grade é atualizada quando o número quântico principal muda. No runner científico, a grade é reaproveitada enquanto $n$ permanece igual na ordem de execução. Cada combinação de orbital e base possui seu próprio cálculo dos campos, que são então utilizados pelos modos de exportação compatíveis.

Na etapa Blender, os módulos distribuem a leitura dos arquivos, a construção dos objetos, a aplicação dos materiais e a exportação FBX. A organização espacial e a hierarquia de coleções são aplicadas à cena salva para inspeção.

No Unity, o componente `OrbitalRecordingRunner` coordena a preparação e a gravação de uma Run. O `OrbitalRunImporter` importa os FBXs e seleciona o catálogo correspondente, enquanto `OrbitalRunSettings` lê, valida e disponibiliza os parâmetros de apresentação e gravação do Job. Após a entrada em Play Mode e a inicialização do apresentador, esses parâmetros são aplicados e o lote é encaminhado ao gravador. A Figura 6 apresenta essa sequência.

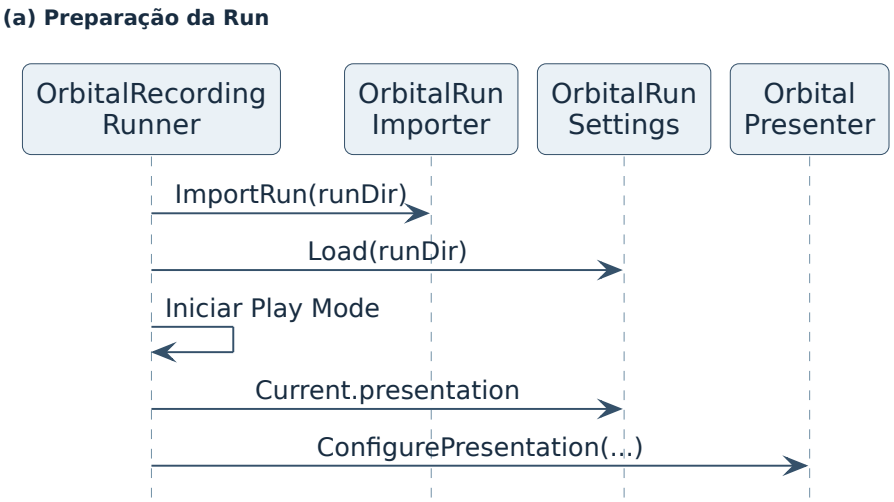



Figura 6: Sequência da Run no Unity. (a) Importação e configuração da apresentação.
Fonte: Elaborado pelo autor

A preparação configura o conjunto de modelos e os parâmetros comuns ao lote. No ciclo de cada gravação necessária, o `OrbitalGifRecorder` solicita ao `OrbitalPresenter` o carregamento do modelo. O apresentador instancia a geometria, realiza o enquadramento e adiciona o componente `Rotation` ao pivô.

O gravador obtém a velocidade e o número de voltas de `Rotation` para calcular a duração pela Eq. (3.8). Os parâmetros de captura são lidos de `OrbitalRunSettings`. Durante a gravação, o Unity executa `Rotation.Update()`, que modifica a orientação do pivô. Ao concluir o lote, o gravador comunica o resultado ao runner.

**(b) Gravação e rotação**

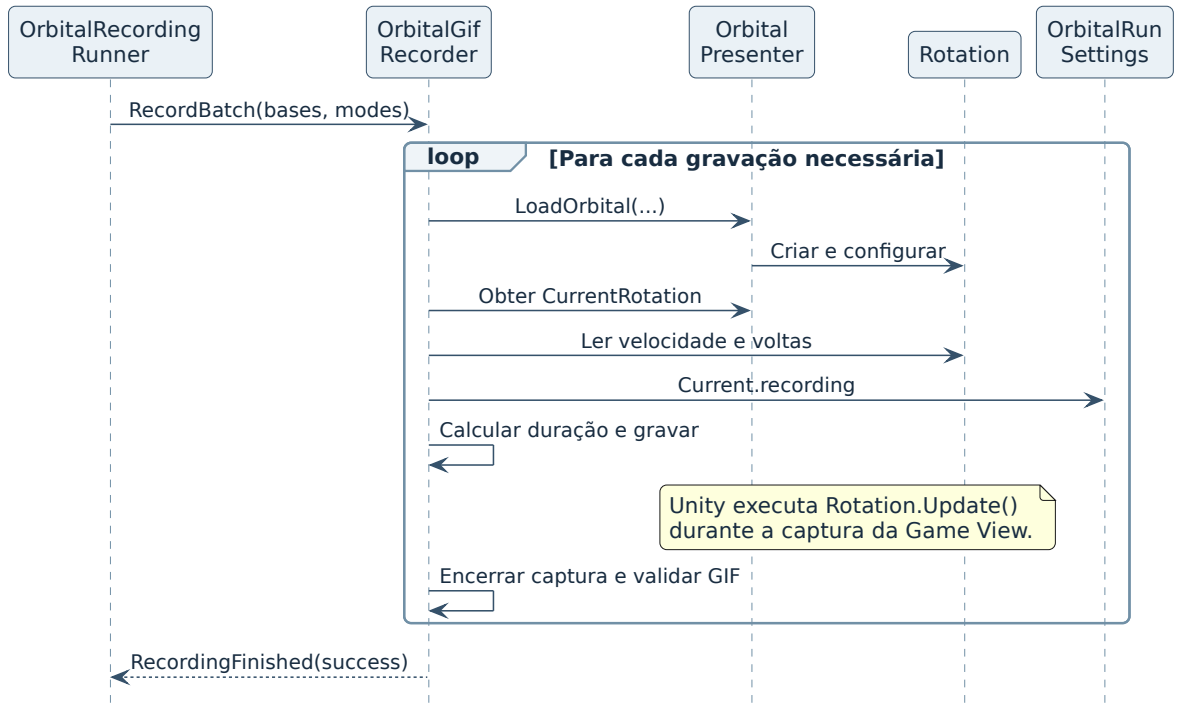



Figura 6: Sequência da Run no Unity (continuação). (b) Gravação do lote e participação do componente de rotação.

Fonte: Elaborado pelo autor

A janela `OrbitalRecordingWindow` permite selecionar bases e representações para a gravação pelo Editor. Separadamente, `OrbitalRunCleanup` implementa a remoção das cópias importadas no projeto Unity, preservando os arquivos originais das Runs.

O enquadramento é realizado pelo `OrbitalPresenter`. Inicialmente, calcula-se uma caixa envolvente que reúne todas as malhas do modelo, e a geometria é deslocada para que o centro dessa caixa coincida com o pivô. Se $\mathbf{e} = (e_x, e_y, e_z)$ contém as semiextensões da caixa, o raio da esfera envolvente é $R = \|\mathbf{e}\| = \sqrt{e_x^2 + e_y^2 + e_z^2}$. A Figura 7 ilustra essa construção.

Na configuração de câmera em perspectiva utilizada, o pivô está sobre o eixo de visão. Sua profundidade axial $d$ corresponde à projeção do vetor entre a câmera e o pivô sobre esse eixo. Se $\alpha_{\mathrm{v}}$ é o semiângulo vertical e $a$ é a razão entre a largura e a altura da imagem, definem-se

$$\alpha_{\mathrm{h}} = \arctan\left(a \tan \alpha_{\mathrm{v}}\right), \qquad \alpha = \min\left(\alpha_{\mathrm{v}}, \alpha_{\mathrm{h}}\right), \tag{4.1}$$

em que $\alpha$ representa o semiângulo limitante do campo de visão.

Pela condição de tangência da esfera aos limites do campo de visão, o raio limite é $d \sin \alpha$. Introduzindo o fator $f = \texttt{screen_fill}$, calculam-se o raio alvo e o fator multiplicativo de escala do pivô:

$$R_{\text{alvo}} = f d \sin \alpha, \qquad s = \frac{R_{\text{alvo}}}{R}. \tag{4.2}$$

Como a rotação preserva as distâncias ao pivô, a esfera continua envolvendo o modelo nas diferentes orientações. O fator $f < 1$ estabelece uma margem em relação ao limite geométrico.

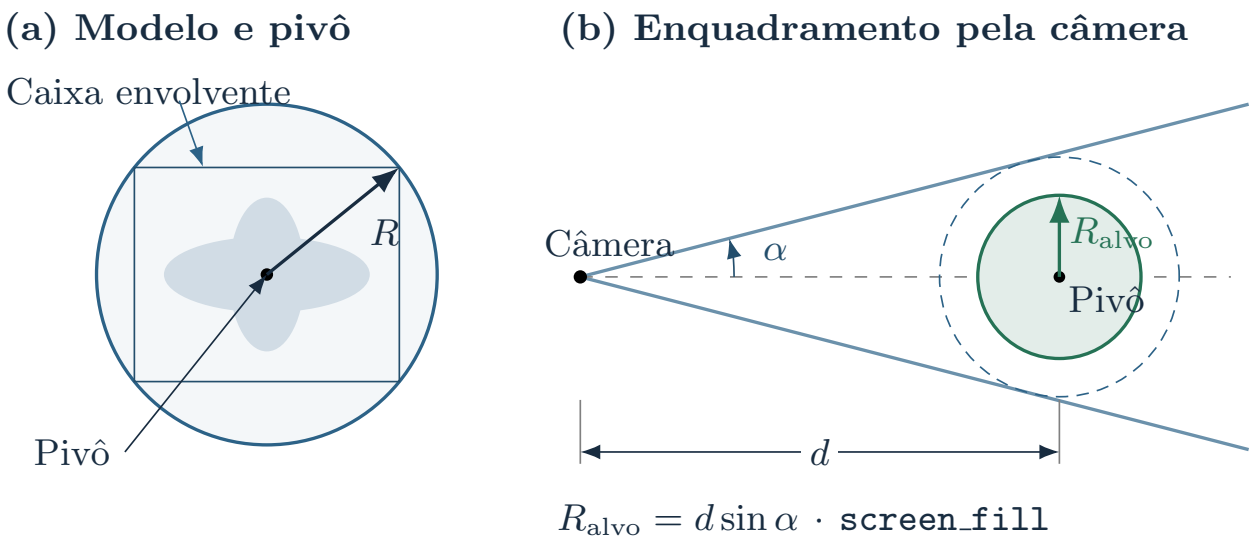



Figura 7: Geometria do enquadramento no Unity. O raio $R$ vem da caixa envolvente, $d$ é a profundidade axial do pivô e $\alpha$ é o menor semiângulo do campo de visão.

Fonte: Elaborado pelo autor

Esse ajuste determina o tamanho do modelo na Game View, capturada pelo Recorder. A comparação da Figura 3 ilustra o efeito de alterar $f$, mantendo a resolução, e o efeito de alterar a resolução, mantendo o enquadramento.

## 4.3 Configuração de Jobs e rastreabilidade das Runs

Os pedidos de geração são definidos por presets armazenados em `config/jobs/`. Cada preset especifica a seleção dos orbitais, as representações e eventuais alterações dos parâmetros padrão de `config/defaults.json`. Na preparação, o runner `prepare_run` combina essas informações, resolve os parâmetros de cada representação e valida a configuração resultante.

Após essa resolução, cria-se uma pasta em `runs/<id>/`. O identificador combina a data e o horário de criação em UTC com um sufixo de oito caracteres hexadecimais. Nesse diretório, são gravados o `job.json`, com a configuração consolidada, e o `run.json`, com o estado inicial da execução. Também são criadas as pastas das três etapas.

A Tabela 3 apresenta a organização dos registros e das saídas. Os caminhos são relativos ao diretório `runs/<id>/`, e `<modo>` identifica a representação utilizada.

Tabela 3: Organização dos arquivos de uma Run.

| Caminho | Conteúdo |
|---|---|
| `job.json` | Configuração efetiva da execução. |
| `run.json` | Identificador, estados, horários e eventual erro da execução. |
| `scientific/<modo>/` | Malhas OBJ ou arquivos com coordenadas dos pontos. |
| `blender/<modo>/` | Modelos FBX preparados no Blender. |
| `blender/<id>.blend` | Cena para inspeção, quando seu salvamento é solicitado. |
| `unity/<modo>/` | Arquivos GIF finalizados. |
| `unity/<modo>/_incomplete/` | Arquivos temporários de gravação. |
| `unity/Editor.log` | Log do Editor na execução pelo runner do Unity. |

Fonte: Elaborado pelo autor

Os runners recebem o caminho da Run e utilizam o `job.json` nela armazenado. Dessa forma, alterações posteriores nos presets ou nos valores padrão não modificam automaticamente a configuração de uma execução já preparada. O `run.json`, por sua vez, é atualizado durante o processamento, registrando a etapa ativa, os estados das etapas, os horários de início e término e eventuais falhas.

As saídas são organizadas por etapa e representação, enquanto seus nomes identificam os números quânticos e a base. Nos GIFs, o nome também inclui o modo. Essa combinação permite associar cada arquivo à configuração e ao estado da execução que o produziu.

Como exemplo, a Run piloto `run_20261104_110942_5de6180b` registra a seleção $(n, \ell, m) = (4, 2, 2)$ nas bases real e complexa, com $250$ pontos por eixo. Seu `run.json` registra a conclusão das três etapas. Nessa pasta, o arquivo `orbital_n4_l2_m2_real_signed.gif` encontra-se em `unity/signed/`, permitindo relacionar a animação aos parâmetros utilizados em sua geração.

O código-fonte, os presets, a documentação e os registros e saídas dessa Run piloto estão disponíveis no repositório do projeto [11]. A organização por Run também fornece os registros utilizados para acompanhar o processamento e orientar a retomada descrita na próxima subseção.

## 4.4 Orquestração, validação e retomada

A orquestração pelo Node-RED organiza as chamadas aos runners em um fluxo de mensagens. As operações são agrupadas em *subflows*, que encapsulam conjuntos de nós em componentes reutilizáveis [13]. No projeto, esses componentes representam a preparação da Run, as três etapas de processamento e a recuperação de execuções interrompidas.

O caminho principal parte do nó *Gerar Job*, que recebe o nome de um preset, e percorre *Preparar Run*, *Científico*, *Blender* e *Unity*. A Figura 8 apresenta esse caminho e as entradas de retomada.

A preparação retorna uma resposta JSON com o identificador e o caminho da Run. Após verificar essa resposta, o fluxo mantém os valores em `msg.run_id` e `msg.run_dir`. O campo `msg.payload` é preparado com os argumentos necessários a cada chamada. Nas etapas científica, Blender e Unity, a mensagem avança somente quando o processo termina com código de saída zero. As saídas normal e de erro são encaminhadas a nós de depuração.

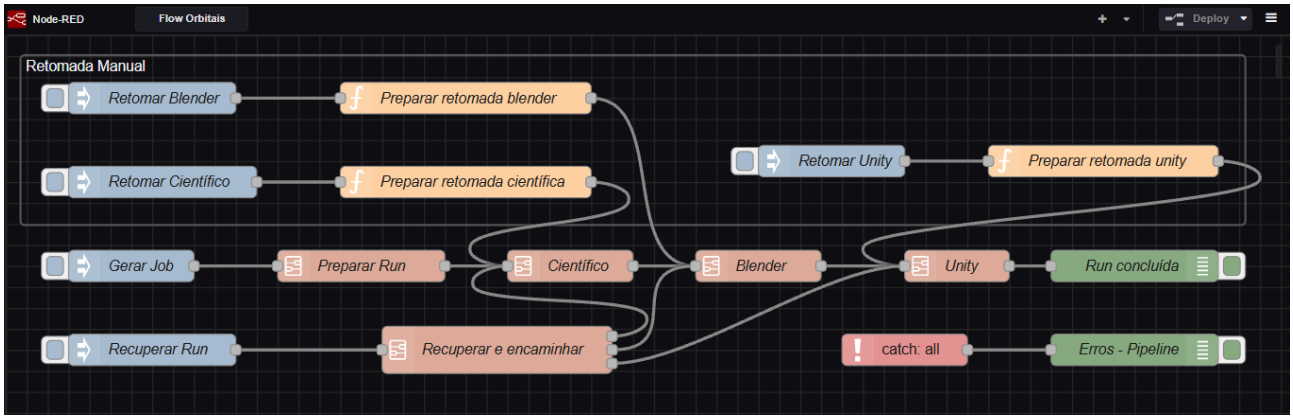



Figura 8: Fluxo de execução e retomada na aba Flow Orbitais do Node-RED.
Fonte: Elaborado pelo autor

As validações são distribuídas entre a preparação e os limites das etapas. Antes da criação da Run, verificam-se a estrutura do pedido, os números quânticos, a compatibilidade entre bases e modos e os valores dos parâmetros. Durante o processamento, os runners conferem as entradas e saídas esperadas. O runner do Blender verifica os arquivos científicos, enquanto o runner do Unity confere os FBXs antes de abrir o Editor e os GIFs após seu encerramento. Essas verificações incluem nomes, presença de conteúdo e correspondência com o conjunto solicitado.

O serviço responsável pelos estados da Run também condiciona o início de uma etapa à conclusão das anteriores. Falhas tratadas pelos runners são registradas no `run.json`, com o estado `failed` e a mensagem de erro. O código de saída impede que o fluxo prossiga normalmente para a etapa seguinte.

Após a correção da causa da falha, os nós *Retomar Científico*, *Retomar Blender* e *Retomar Unity* permitem informar o identificador da execução existente. Essas entradas acionam o runner correspondente com a opção `--resume`. O serviço verifica se a Run está com falha na etapa indicada e se os estados das demais etapas permitem a retomada.

Nas etapas científica e Blender, a retomada limpa as saídas da etapa com falha e refaz seu processamento integralmente. No Unity, preservam-se os GIFs finalizados que satisfazem as verificações implementadas. O gravador percorre os modelos solicitados, reaproveita esses arquivos e grava os restantes. O critério de reaproveitamento considera o tamanho mínimo, a assinatura do formato e o marcador de encerramento do GIF.

Durante a captura, o arquivo é produzido no diretório `_incomplete`. Ao terminar a gravação e passar pela verificação, ele é movido para a pasta final da representação. Essa organização permite distinguir as saídas finalizadas dos arquivos temporários.

Uma interrupção abrupta pode deixar a Run registrada como `running`, sem que o processo continue ativo. Nesse caso, o nó *Recuperar Run* exige a confirmação do operador de que a execução anterior terminou. O componente *Recuperar e encaminhar* identifica a etapa interrompida, registra sua falha e encaminha a mesma Run à entrada de retomada correspondente.

Quando a interrupção ocorre entre etapas e a próxima permanece como `pending`, essa etapa pode ser acionada pelo terminal sem `-resume`. A Run recebe o estado `completed` quando todas as etapas registram sua conclusão.

# 5 RESULTADOS E DISCUSSÃO

Esta seção apresenta os resultados da geração automatizada dos orbitais e de suas animações. Inicialmente, verifica-se a cobertura do lote completo, comparando as quantidades de arquivos produzidos com as previstas para cada base e representação. Em seguida, utiliza-se o orbital $(n, \ell, m) = (4, 2, 2)$ da execução piloto para comparar visualmente os modos e as bases. Por fim, examinam-se diferentes vistas de uma mesma animação e discutem-se as possibilidades e as limitações das representações para a visualização dos orbitais.

## 5.1 Execução em lote e cobertura das representações

A execução do lote completo foi registrada na Run `run_20261104_114137_42180c82`, com os parâmetros apresentados na Tabela 2. O arquivo `run.json` registra o estado `completed` para as etapas científica, Blender e Unity, bem como para a execução como um todo. A conferência das saídas segue os critérios da subseção 3.5.

Para cada número quântico principal $n$, a quantidade de orbitais espaciais em cada base é dada por

$$N_n = \sum_{\ell=0}^{n-1} (2\ell + 1) = n^2, \tag{5.1}$$

pois cada valor de $\ell$ admite $2\ell + 1$ valores de $m$. Em cada base, foram solicitados quatro modos compatíveis: os três modos comuns às duas bases e, adicionalmente, `signed` na base real ou `phase` na complexa. A quantidade prevista de animações para $n = 1, \ldots, 4$ é, portanto,

$$N_{\mathrm{GIF}} = \sum_{n=1}^{4} 8n^2 = 8(1 + 4 + 9 + 16) = 240. \tag{5.2}$$

A Tabela 4 apresenta a contagem dos arquivos finais, agrupados por base, representação e número quântico principal.

A contagem obtida corresponde à previsão em todos os grupos, totalizando $240$ GIFs finais. Os arquivos contabilizados eram não vazios e apresentavam nomes compatíveis com os números quânticos, a base e o modo solicitados. Os arquivos temporários do diretório `_incomplete` foram excluídos dessa contagem.

Cada combinação de base e modo reúne $30$ animações, distribuídas entre os quatro valores de $n$. Esse total contabiliza arquivos por rótulo $(n, \ell, m)$, base e representação. A correspondência entre os

Tabela 4: Contagem dos GIFs finais do lote por número quântico principal, base e modo de representação.

| Base | Modo | $n = 1$ | $n = 2$ | $n = 3$ | $n = 4$ | Total |
|---|---|---|---|---|---|---|
| Real | `single` | 1 | 4 | 9 | 16 | 30 |
| Real | `signed` | 1 | 4 | 9 | 16 | 30 |
| Real | `multilayer` | 1 | 4 | 9 | 16 | 30 |
| Real | `point_cloud` | 1 | 4 | 9 | 16 | 30 |
| Complexa | `single` | 1 | 4 | 9 | 16 | 30 |
| Complexa | `multilayer` | 1 | 4 | 9 | 16 | 30 |
| Complexa | `point_cloud` | 1 | 4 | 9 | 16 | 30 |
| Complexa | `phase` | 1 | 4 | 9 | 16 | 30 |
| **Total** | | 8 | 32 | 72 | 128 | 240 |

Fonte: Elaborado pelo autor

conjuntos esperado e produzido demonstra a cobertura das combinações solicitadas na configuração testada.

## 5.2 Comparação das representações e das bases

A Figura 9 apresenta o orbital $(n, \ell, m) = (4, 2, 2)$ da execução piloto nos oito pares compatíveis de base e representação. Os quadros correspondem ao instante de $0{,}50$ s das animações, com a mesma orientação de apresentação e recorte idêntico. Mantêm-se os parâmetros comuns de geração e gravação, enquanto cada modo utiliza os critérios geométricos e de cor descritos na seção de métodos.

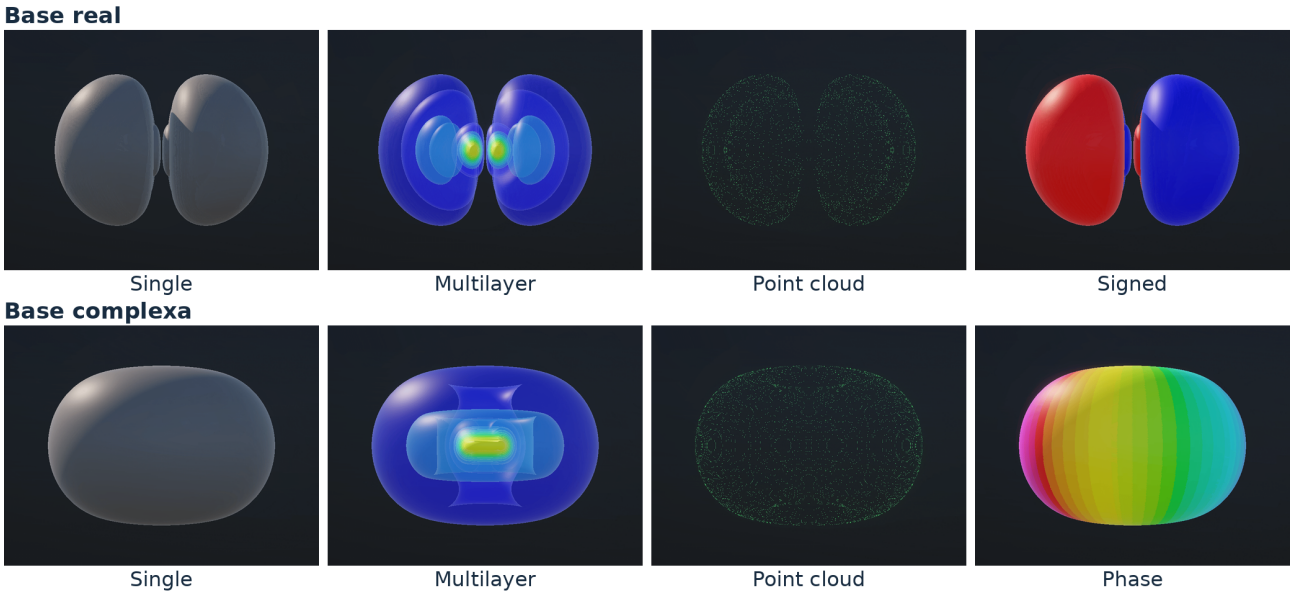



Figura 9: Representações de $(n, \ell, m) = (4, 2, 2)$ nas bases real e complexa. Quadros em $0{,}50$ s das animações da Run piloto, com recorte idêntico.

Fonte: Elaborado pelo autor

Na base real, o modo `single` evidencia a separação dos lobos visíveis nessa orientação, apresentando uma superfície de cor uniforme. O modo `signed` preserva essa estrutura geral e acrescenta

a distinção entre regiões de sinais opostos. As cores vermelha e azul representam, respectivamente, os sinais positivo e negativo segundo a convenção adotada.

O modo `multilayer` apresenta superfícies em diferentes níveis de densidade. A transparência das camadas permite observar estruturas internas que ficam encobertas pela superfície externa no modo `single`. Nesse caso, as cores identificam níveis de densidade. No modo `point_cloud`, os pontos acompanham a distribuição espacial dos lobos, com uma aparência menos contínua que a das malhas triangulares. Sua disposição resulta da seleção de pontos da grade na faixa de densidade estabelecida.

Na base complexa, a superfície externa apresenta um contorno mais contínuo nessa vista. O modo `multilayer` evidencia camadas internas distribuídas ao redor da região equatorial, enquanto `point_cloud` acompanha a mesma estrutura geral por meio dos pontos selecionados. O modo `phase` mantém a isosuperfície de densidade e acrescenta faixas de cores associadas à variação espacial da fase. As fronteiras dessas faixas decorrem da discretização da escala de cores.

As diferenças entre as bases podem ser interpretadas pela dependência angular dos harmônicos. Para $m = 2$, o harmônico complexo contém o fator $e^{2i\phi}$, cujo módulo é unitário. Sua densidade é, portanto, independente do ângulo azimutal. A combinação real definida na Eq. (2.8) introduz uma dependência proporcional a $\cos(2\phi)$ na função de onda [3]. Para as funções analíticas selecionadas, obtém-se

$$\rho_{\mathrm{real}}(r, \theta, \phi) = 2\, \rho_{\mathrm{complexa}}(r, \theta) \cos^2(2\phi). \tag{5.3}$$

Essa modulação azimutal explica a formação de lobos na representação real e a simetria em torno do eixo polar da densidade complexa.

A comparação seleciona vetores diferentes nas duas bases sob o mesmo rótulo $(4, 2, 2)$. Ambos permanecem autoestados de energia, mas possuem distribuições espaciais distintas. A reexpressão de um mesmo estado em outra base, com a transformação correspondente dos coeficientes, preservaria sua densidade.

Também é necessário considerar que o limiar relativo utiliza o máximo de cada campo e que o enquadramento é ajustado individualmente. A comparação do painel é, assim, qualitativa e centrada na forma e na informação visualizada. Os modos de densidade mostram sua distribuição espacial em diferentes níveis, enquanto `signed` e `phase` acrescentam informações de sinal e fase que o módulo ao quadrado elimina.

## 5.3 Animações e limitações da visualização

A Figura 10 apresenta três quadros da animação do orbital $(n, \ell, m) = (4, 2, 2)$, na base real e no modo `signed`, provenientes da execução piloto. Com velocidade angular de $90^\circ/\mathrm{s}$ e três voltas, a duração programada é de $12$ s, conforme a Eq. (3.8). Os instantes de $0{,}50$, $4{,}50$ e $8{,}50$ s correspondem aos trechos de rotação em torno dos eixos $Y$, $X$ e $Z$, respectivamente.

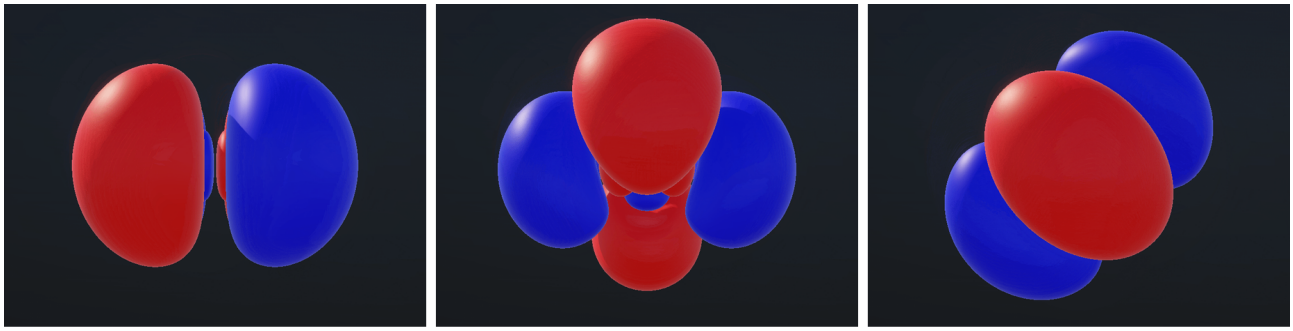



Figura 10: Quadros de uma mesma animação de $(n, \ell, m) = (4, 2, 2)$ na base real, no modo `signed`, durante a rotação do modelo.

Fonte: Elaborado pelo autor

No primeiro quadro, dois lobos externos predominam na projeção, enquanto partes menores aparecem na região central. No segundo, a orientação permite distinguir mais claramente os lobos vermelhos acima e abaixo e os azuis nas laterais. No terceiro, a sobreposição entre as partes do modelo evidencia a dependência da aparência em relação à orientação. A sequência permite, portanto, observar estruturas que ficam parcialmente encobertas em uma imagem isolada.

O movimento é uma operação de apresentação aplicada à geometria. A densidade representada corresponde a um estado estacionário e permanece constante no tempo, conforme a Eq. (2.3). As cores acompanham as regiões de sinal da função espacial durante a rotação. Assim, a animação mostra diferentes orientações do mesmo modelo.

Cada quadro do GIF é uma projeção bidimensional da geometria tridimensional. A rotação fornece diferentes vistas e indícios de profundidade, mas a oclusão ainda limita a observação simultânea das estruturas internas. No modo `signed`, a opacidade das superfícies torna essa sobreposição evidente. A transparência de `multilayer` oferece outra forma de examinar o interior, conforme o painel da Figura 9.

A geometria apresentada também depende do domínio amostrado, do espaçamento da grade e dos limiares escolhidos. As isosuperfícies delimitam níveis selecionados do campo, e sua extensão visual depende desses parâmetros. A interpretação deve considerar que a função de onda é definida no espaço contínuo, enquanto o modelo utiliza uma amostragem finita.

Na etapa de apresentação, o enquadramento individual favorece a visualização de cada modelo, mas exige cuidado em comparações de tamanho entre orbitais. A resolução da captura e a codificação do GIF também condicionam a reprodução de detalhes e das escalas de cores. As comparações realizadas permitem examinar qualitativamente a forma, a distinção entre regiões e as mudanças de orientação nas animações produzidas.

Os GIFs constituem o material destinado à visualização no projetor. Os resultados documentados nesta seção referem-se à geração e à inspeção das animações no ambiente computacional. A resolução máxima do projetor limita o nível de detalhe que pode ser reproduzido na exibição.

# 6 CONCLUSÕES E CONSIDERAÇÕES FINAIS

Este trabalho desenvolveu um fluxo computacional para gerar modelos tridimensionais de orbitais do hidrogênio e animações GIF destinadas à visualização em projetores. A integração entre Python, Blender e Unity, coordenada pelo Node-RED, reuniu o cálculo dos campos, a preparação das geometrias e a gravação das animações em execuções com parâmetros e estados registrados.

Foram implementados cinco modos de representação, que permitem visualizar níveis de densidade, o sinal das funções reais e a fase das funções complexas. O lote para $n = 1, \ldots, 4$ produziu os $240$ GIFs previstos pelas combinações de números quânticos, bases e modos compatíveis. A conferência das saídas e dos registros confirmou a cobertura solicitada e a conclusão das etapas na configuração testada.

A comparação do orbital $(4, 2, 2)$ mostrou que os modos oferecem informações complementares sobre as funções espaciais selecionadas. As diferenças entre as funções selecionadas nas bases real e complexa foram relacionadas à sua dependência angular. A rotação dos modelos permitiu observar estruturas parcialmente encobertas em vistas isoladas, preservando a distinção entre a apresentação geométrica e a evolução temporal de um estado quântico.

O alcance da avaliação compreende a geração dos arquivos e sua inspeção no ambiente computacional. A amostragem finita, os limiares e as condições de apresentação influenciam a aparência das representações. Como desdobramentos, propõem-se a avaliação da reprodução e da legibilidade em função da resolução do projetor, a investigação da sensibilidade à resolução e aos limiares e a avaliação do uso das animações em atividades com o público. O código e a organização das Runs fornecem uma base para ampliar essas verificações e preparar novos conjuntos de visualizações.

# REFERÊNCIAS


[1] Matthew D. Hanson. Visualizing the hydrogen atomic orbitals: A tool for undergraduate physical chemistry. *Journal of Chemical Education*, 101(8):3539–3546, 2024. DOI 10.1021/acs.jchemed.4c00547.

[2] Fernando Bermude Graça. Mecânica quântica em realidade virtual. Trabalho de Conclusão de Curso, Instituto de Física de São Carlos, Universidade de São Paulo, São Carlos, 2024.

[3] David Jeffrey Griffiths. *Mecânica quântica*. Pearson Prentice Hall, São Paulo, second edition, 2011. Tradução de Lara Freitas. Revisão técnica de Marcelo Mulato.

[4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. DOI 10.1038/s41586-020-2649-2.

[5] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. DOI 10.1038/s41592-019-0686-2.

[6] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, Tony Yu, and the scikit-image contributors. scikit-image: Image processing in Python. *PeerJ*, 2:e453, 2014. DOI 10.7717/peerj.453.

[7] Blender Foundation. FBX. Manual do Blender. https://docs.blender.org/manual/pt/5.1/files/import_export/fbx.html. Acesso em 4 nov. 2026.

[8] Unity Technologies. Unity 6.5 User Manual. https://docs.unity3d.com/6000.5/Documentation/Manual/UnityManual.html. Acesso em 4 nov. 2026.

[9] Unity Technologies. Movie Recorder. Manual do Unity Recorder 5.1.7. https://docs.unity3d.com/Packages/com.unity.recorder@5.1/manual/RecorderMovie.html. Acesso em 4 nov. 2026.

[10] OpenJS Foundation and Contributors. Node-RED. Software, versão 5.0.9. https://nodered.org/. Acesso em 4 nov. 2026.

[11] Thiago Oliveira dos Santos. Orbitais-Blender. Repositório de código-fonte no GitHub. https://github.com/tubar02/Orbitais-Blender, 2026. Acesso em 4 nov. 2026.

[12] Thomas Lewiner, Hélio Lopes, Antônio Wilson Vieira, and Geovan Tavares. Efficient implementation of Marching Cubes' cases with topological guarantees. *Journal of Graphics Tools*, 8(2):1–15, 2003. DOI 10.1080/10867651.2003.10487582.

[13] Node-RED. Subflows. Guia do usuário. https://nodered.org/docs/user-guide/editor/workspace/subflows. Acesso em 4 nov. 2026.